AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
i-Credit Hours Engineering Programs

CSE485-CSE488: Deep Learning

Assignment 1

MicroGrad
Scalar Autograd Engine

Yousef Mohamed Mahmoud
Helmy Ismail

Student ID: **20P7954**

Git branch
`Yousef-Mohamed-Mahmoud-Helmy-Ismail_20P7954`

View submitted code on GitHub

3 November 2026

Scope. A standard-library scalar differentiation engine, a composable neural-network API, independent correctness checks, and an executed two-moons classification notebook. All gradients used for training are computed by the submitted engine.
Verified result. 37 local tests pass. Held-out accuracy is 97.50% after 300 full-batch updates.

1 Chain rule and primitive arithmetic

Specification coverage. The supplied three-page assignment requires `Value`, arithmetic and activation derivatives, DFS topological sorting, `Neuron/Layer/MLP`, a demo notebook, and a LaTeX report. There are no additional standalone conceptual questions in the PDF or template README. The explanations below answer the conceptual issues implicit in those requirements, including both TA warnings.

1.1 Reverse-mode differentiation on a DAG

Let L be a scalar output, and let $C(u)$ contain the immediate consumers of u in the forward computational graph. These consumers are sometimes called the parents of u when tracing backward. The multivariable chain rule is

$$\frac{\partial L}{\partial u} = \sum_{v \in C(u)} \frac{\partial L}{\partial v} \frac{\partial v}{\partial u}. \tag{1}$$

Each summand describes one path by which changing u changes L. Repeated operand occurrences contribute separately even if the graph stores a single shared input node. The terminal seed is $\partial L/\partial L = 1$. Reverse mode is useful here because one backward sweep yields the derivative of a scalar loss with respect to every network parameter.

1.2 Addition

For $z = x + y$, holding y fixed gives $\lim_{h\to 0}[(x + h + y) - (x + y)]/h = 1$. By symmetry,

$$\frac{\partial z}{\partial x} = 1, \qquad \frac{\partial z}{\partial y} = 1.$$

If $\bar{z} = \partial L/\partial z$, add $\bar{z}$ to both operand gradients. For $z = x + x$, the same node receives two increments, giving $2\bar{z}$.

1.3 Multiplication

For $z = xy$, the difference quotient with respect to x is $[(x + h)y - xy]/h = y$; interchanging x and y yields

$$\frac{\partial z}{\partial x} = y, \qquad \frac{\partial z}{\partial y} = x.$$

The closure therefore adds $y\bar{z}$ to x and $x\bar{z}$ to y. For $z = x^2$ implemented as $x * x$, both increments reach x and sum to $2x\bar{z}$.

1.4 Constant powers and derived operations

For a fixed real k, $z = x^k$ has local derivative kx^{k-1} on its differentiable real domain. For positive integer k, this follows by expanding $(x + h)^k$ and retaining the coefficient of h. Negative and fractional powers follow the usual extension on valid real domains.

Negation is multiplication by -1; subtraction is addition of a negation. Division is $x/y = x\,y^{-1}$, so $\partial(x/y)/\partial x = 1/y$ and $\partial(x/y)/\partial y = -x/y^2$. Reverse operators allow expressions such as $2 - x$ and $2/x$ to construct the same differentiable primitives.

2 Activations and numerical conventions

2.1 Exponential

For $z = e^x$, using $\lim_{h\to 0}(e^h - 1)/h = 1$ gives

$$\frac{dz}{dx} = \lim_{h\to 0} e^x \frac{e^h - 1}{h} = e^x = z.$$

The backward closure reuses the forward result: $\bar{x}$ += $z\bar{z}$. The implementation calls `math.exp`; sufficiently large positive inputs raise Python's `OverflowError` instead of silently fabricating a value.

2.2 Hyperbolic tangent

Writing $q = e^{2x}$ gives $z = (q-1)/(q+1)$ and $dq/dx = 2q$. The quotient rule yields

$$\frac{dz}{dx} = \frac{2q(q+1) - (q-1)2q}{(q+1)^2} = \frac{4q}{(q+1)^2},$$
$$1 - z^2 = \frac{(q+1)^2 - (q-1)^2}{(q+1)^2} = \frac{4q}{(q+1)^2}.$$

Thus $dz/dx = 1 - z^2$ and $\bar{x}$ += $(1 - z^2)\bar{z}$. The code uses `math.tanh(x)` for the forward pass, avoiding the overflow-prone explicit exponential formula. Inputs ± 1000 are checked: the outputs saturate at ± 1 and their floating-point derivatives are zero.

2.3 Optional ReLU

For $z = \max(0, x)$ the derivative is 1 for $x > 0$ and 0 for $x < 0$. At the nondifferentiable point $x = 0$, the implementation chooses subgradient zero. ReLU is included as a supplemental operation; the template networks continue to use tanh at every layer.

2.4 Explicit domain and state decisions

- Values wrap Python `int` or `float` data. Strings, lists, and other unsupported data raise `TypeError`.
- Exponents are fixed `int`/`float` scalars, matching the template assertion contract. Differentiable exponents are outside scope.
- Division by zero raises `ZeroDivisionError`. Powers producing complex results are rejected because this engine represents real scalars.
- x^0 is treated as a constant with derivative zero, including Python's forward convention $0^0 = 1$. A fractional power at zero may have no finite derivative; the implementation does not promise differentiation there.
- Data must remain unchanged between graph construction and backward. Training updates parameters only after the entire backward sweep.

The engine performs ordinary Python floating-point arithmetic. It does not provide symbolic simplification, automatic precision selection, or special handling for every nonfinite input.

3 Value architecture and gradient lifecycle

Field	Purpose
`data`	Scalar forward value.
`grad`	Accumulated derivative of the chosen scalar output. Initially 0.0.
`_prev`	Set of input-node references; keeps dependencies for traversal.
`_backward`	Closure implementing one operation's local chain rule.
`_op`, `label`	Operation and human-readable label for inspection.

Each overloaded operator wraps scalar constants, computes a forward scalar, creates a new output node, and installs a closure referring to its operands and output. Constants also receive gradients in this simple design; only the list returned by `parameters()` is updated during optimization.

3.1 Why a set does not erase repeated-operand derivatives

For $c = a + a$, `c._prev` contains a once. This is sufficient for traversal. The addition closure retains both operand roles and executes two increments, even though `self` and `other` reference the same object. Graph-node identity and operand multiplicity serve different purposes.

```
def _backward():
    self.grad += out.grad
    other.grad += out.grad
```

Replacing either increment with assignment loses contributions from other paths. Equation 1 requires a sum; `+=` directly implements it. Only the root seed uses assignment: `self.grad = 1.0`.

3.2 Fresh graphs and deliberate reset semantics

`reset_grad()` clears one Value. `Module.zero_grad()` clears the trainable leaves exposed by that module. Neither automatically resets every intermediate in an old graph. The demo constructs a fresh graph for every loss, zeros parameter gradients, runs backward, and then changes parameter data.

Calling backward twice on the same graph without clearing intermediates can repropagate stale accumulated gradients. To differentiate that same graph again, reset every node returned by `_get_topo()` first. The tests verify both a complete graph reset and deliberate accumulation across fresh graphs sharing a leaf. This behavior is documented in the API rather than silently clearing user gradients inside backward.

3.3 Costs and limits

DFS traversal and reverse execution each take $O(V + E)$ work for a graph with V nodes and E dependency edges, plus $O(V)$ traversal storage. Scalar networks create many Python objects, so this implementation is educational rather than a high-throughput tensor framework. Recursive DFS is limited by Python's recursion depth on very deep chains. Set iteration may choose different valid sibling orders; all still place each dependency before its consumer.

4 Topological sorting and the two traps

```
def _get_topo(self):
    topo, visited = [], set()
    def build_topo(node):
        if node not in visited:
            visited.add(node)
            for child in node._prev:
                build_topo(child)
            topo.append(node)  # outside the child loop
    build_topo(self)
    return topo
```

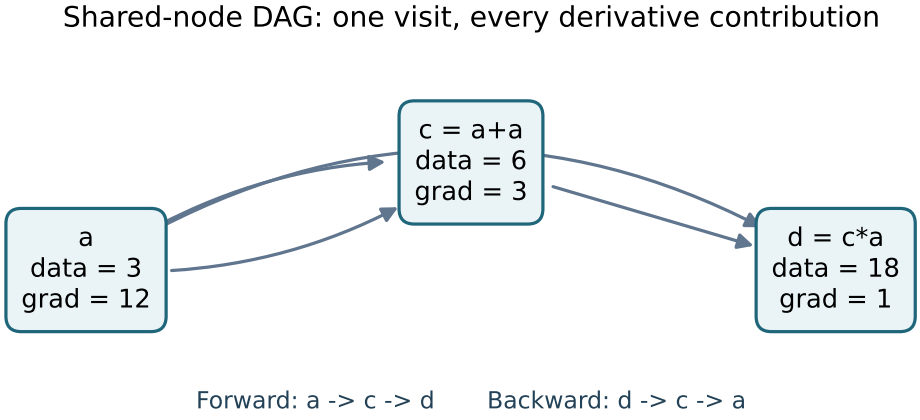
`backward()` seeds the root with 1.0, then runs the closures in reverse topological order.

4.1 The Post-Order Trap

Forward topological order places an operand before each consumer. Appending only after visiting *all* inputs establishes that invariant inductively: leaves are appended immediately, and every internal node is appended after its already-ordered dependencies. Reversing the list places every consumer before its operands, so all upstream contributions arrive before a node propagates them further. Appending inside the child loop can insert a node prematurely or repeatedly; appending only inside that loop also omits leaves.

4.2 The Diamond Graph Trap

Consider $c = a + a$ and $d = ca = 2a^2$. At $a = 3$, $c = 6$ and $d = 18$. Backpropagation first adds $a = 3$ to c's gradient and $c = 6$ directly to a's gradient. Then c contributes 3 twice to a, yielding $\bar{a} = 6 + 3 + 3 = 12$. The graph below shows repeated operand edges explicitly.



One shared Value, two operand roles, three total gradient contributions.

The visited set ensures each unique node appears once, preventing redundant execution on shared subgraphs. A genuine DAG cannot loop infinitely merely because a node is reused; revisiting causes duplicate work, potentially exponential traversal, and incorrect propagation if closures run repeatedly. The assignment warning motivates deduplication, but cycles are not created by ordinary forward operator construction.

5 Neural-network API and training design

5.1 Composition and parameter ownership

A shared `Module` supplies `zero_grad()` through `parameters()`. A `Neuron(nin)` owns `nin` weights and one bias, all initialized independently from $\mathcal{U}(-1, 1)$. Its output is

$$a = \tanh\left(\sum_{j=1}^{n_{in}} w_j x_j + b\right).$$

A `Layer(nin,nout)` evaluates independent neurons on the same input. It returns a list for multiple outputs and a scalar Value for a single output, as required by the template. `MLP` chains these layers. When an intermediate layer has width one, it wraps that scalar in a one-element list before the next layer. This avoids a hidden shape failure in bottleneck architectures such as $2 \to 1 \to 3 \to 1$.

Parameter collection flattens neuron, layer, and model ownership in a stable order, without cloning the Value objects. Gradients and updates therefore refer to the same underlying weights. Positive dimensions and exact input widths are validated so `zip` cannot silently discard missing inputs.

5.2 Reproducible experiment

The executed notebook creates two interleaving semicircles with Gaussian coordinate noise $\sigma = 0.12$. There are 240 points and two features. A seeded, class-balanced split uses 160 for training and 80 for testing. Both the Python parameter RNG and NumPy data RNG use seed 42. No normalization statistics, gradients, early stopping decisions, or parameter updates use the test set.

The network $2 \to 8 \to 8 \to 1$ has $(2+1)8 + (8+1)8 + (8+1)1 = 105$ trainable scalars, of which 88 are weights. Every layer uses tanh to preserve the template contract; class labels are -1 and $+1$, and scores are thresholded at zero. The loss is

$$L = \frac{1}{160}\sum_{i=1}^{160}(f_\theta(x_i) - y_i)^2 + \frac{10^{-4}}{88}\sum_{w \in W} w^2.$$

Biases are excluded from regularization. The gradient of the data term with respect to score s_i is $2(s_i - y_i)/160$; the weight penalty contributes $2 \cdot 10^{-4} w/88$ to each regularized weight.

5.3 Optimization lifecycle

For updates $e = 0, \ldots, 299$, use $\eta_e = 0.12(1 - 0.75e/300)$. Each iteration computes predictions and scalar loss, calls `model.zero_grad()`, runs `loss.backward()`, and applies $\theta \leftarrow \theta - \eta_e \nabla L$. A fresh forward graph is then constructed. Metrics are recorded at parameter states 0 through 300, so the last curve point and saved weights describe the same model.

Logging and dense contours evaluate the learned scalar equations without allocating autodiff graphs. This inference path is checked against the actual MLP on all 240 dataset points to 10^{-12} tolerance. All optimization gradients still use Value and its chain-rule closures.

6 Independent correctness evidence

The original template tests and workflow are unchanged. The local suite passes **37/37** cases: 12 original tests and 25 supplemental cases. Supplemental coverage includes repeated operands, topological edge ordering and uniqueness, reverse division/subtraction, fractional and negative powers, stable tanh, ReLU conventions, reset semantics, invalid shapes, all-parameter finite differences for a bottleneck MLP, and a loss-reduction training check.

6.1 Central finite differences

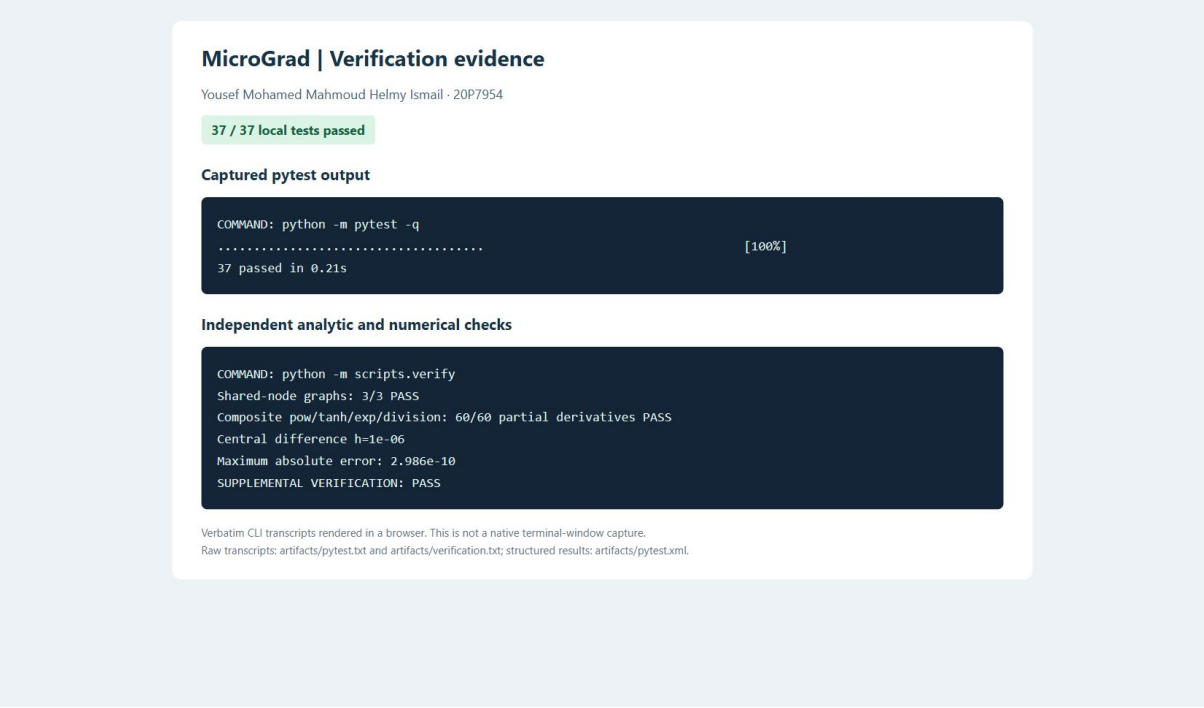
An independent script checks

$$f(x, y) = \tanh(xy + x^3) + \frac{e^{x-y}}{y^2 + 2} + \frac{2}{x^2 + 1}$$

at 30 seeded points in $[-1, 1]^2$. For each coordinate,

$$\frac{\partial f}{\partial x} \approx \frac{f(x + h, y) - f(x - h, y)}{2h}, \qquad h = 10^{-6}.$$

The real-valued reference uses Python math independently of the engine. All 60 partials meet $|g - \hat{g}| < 10^{-7} + 10^{-6}|\hat{g}|$; the largest observed absolute error is 2.986×10^{-10}. Three analytic shared-node checks also pass. The finite-difference formula has $O(h^2)$ truncation error on smooth inputs; very small h amplifies floating-point cancellation. These checks deliberately avoid singularities and ReLU's kink.

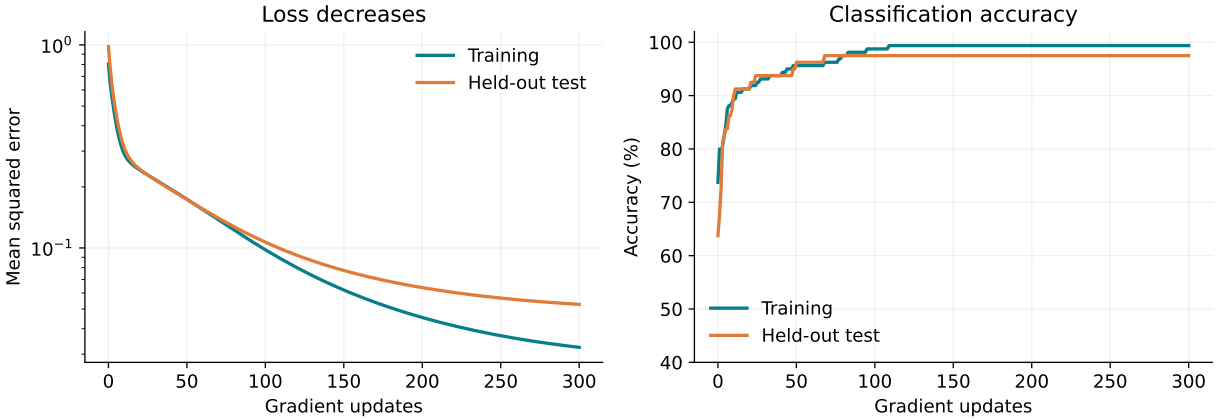


Browser screenshot of verbatim captured CLI output. Raw logs and JUnit XML accompany the submission.

This evidence view displays actual subprocess output rather than simulated results. Terminal UI automation is restricted in this environment; the CLI logs were rendered locally in a browser for screenshot capture. This is a browser capture of the console transcript, not a native terminal-window screenshot.

7 Experimental results

Metric	Before training	After 300 updates
Training objective	0.80365	0.03246
Training data MSE	0.80362	0.03241
Held-out data MSE	0.98049	0.05276
Training accuracy	73.75%	99.38%
Held-out accuracy	63.75%	97.50%



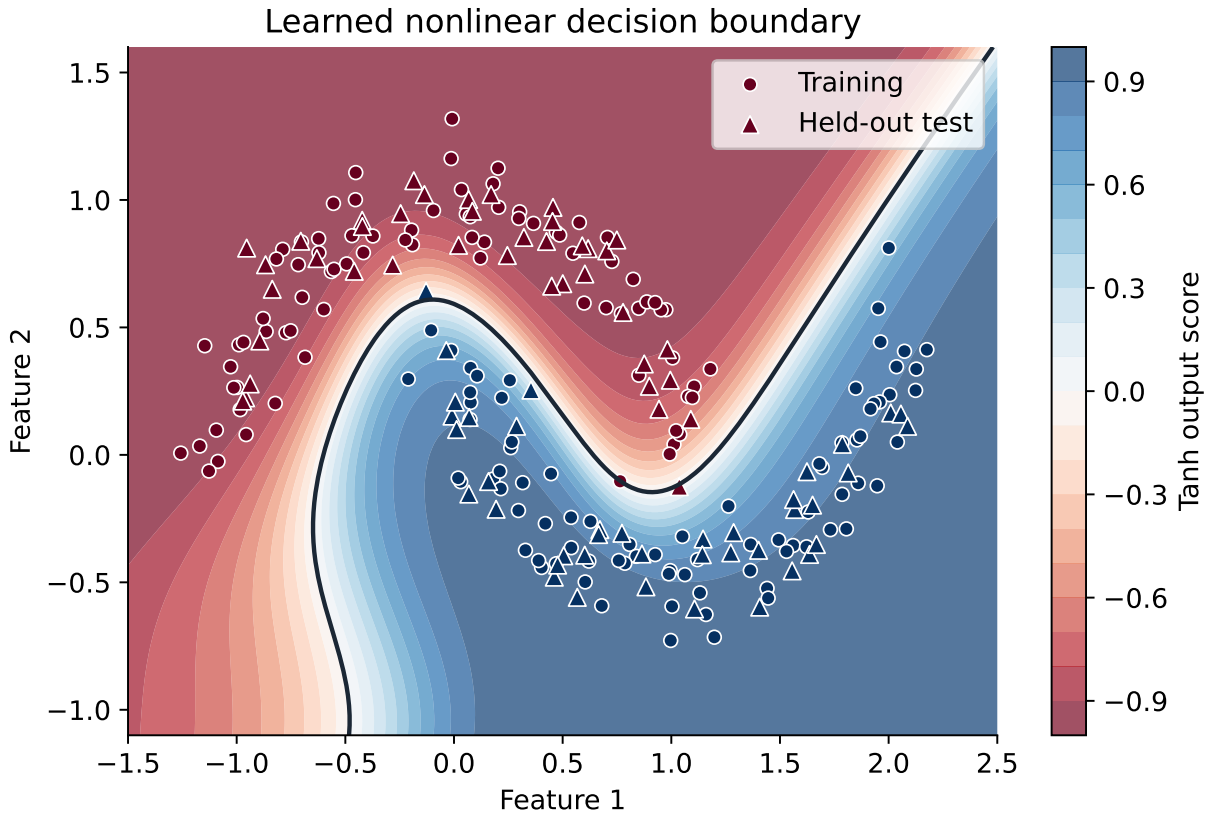
Both data loss and accuracy are measured at the same recorded parameter states.

The training objective decreases from 0.80365 to 0.03246. Held-out accuracy reaches 97.50%, or 78 correctly classified samples out of 80. The test split is balanced, so an always-one-class classifier would score 50%. The curves show how optimization changes fit and classification, while the decision boundary on the next page makes the nonlinear geometry explicit.

Interpretation. A few hundred scalar gradient updates are enough for this compact network to fit the curved structure. MSE measures confidence as well as classification: correctly signed scores can still contribute error when their magnitude differs from the target. The small L2 term discourages large weights but is not evidence of a universally optimal regularization strength.

Limits. This is a single seeded synthetic experiment, not a benchmark across datasets or repeated random trials. Finite samples and label-preserving noise limit the conclusions about generalization. Test curves are reported for inspection; the schedule and number of updates are fixed in advance.

8 Learned decision geometry



Circles are training points; triangles are reserved test points. The dark contour is score zero.

The score field is a composition of tanh activations and learned affine maps. Its curved zero contour separates the interlocking arcs, which a single straight line cannot fully describe. The color scale conveys output score, not a calibrated probability. Outside the sampled arcs the contour is an extrapolation; an attractive plot alone does not establish reliability there.

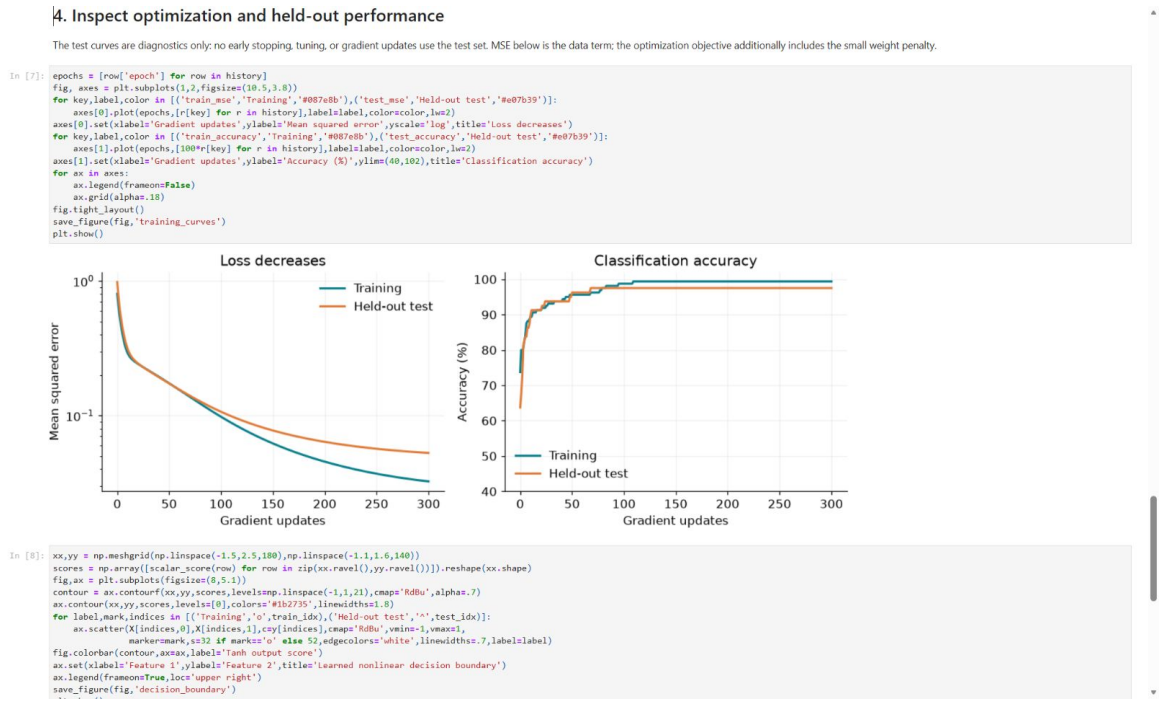
8.1 Why nonlinear layers matter

Without activations, composing affine maps produces another affine map, $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$. Hidden depth by itself would then fail to add nonlinear decision geometry. Tanh breaks this collapse and supplies smooth derivatives, allowing the scalar chain rule to train multiple layers together. Saturation can make those derivatives small, which is one reason to examine both loss and accuracy during learning.

8.2 Reproducible outputs

The executed notebook embeds the graph, learning curves, decision plot, and printed metrics. The exact generated data and train/test indices are saved in `artifacts/dataset.npz`; learned scalars, per-update history, final metrics, and verification details are also exported in readable JSON. Every plotted figure is supplied as a 300 DPI PNG and a vector PDF.

9 Executed notebook and submission reproduction



The locally rendered, executed notebook: recorded outputs and figures are embedded in the saved file.

From the repository root, with Python 3.12 for the pinned notebook environment:

```
python -m venv .venv
# Activate the environment using the command for your platform.
python -m pip install -r requirements.txt
python -m pytest
python -m scripts.verify
python -m nbconvert --to notebook --execute --inplace notebooks/demo.ipynb
python -m nbconvert --to html notebooks/demo.ipynb
```

The report source is compatible with `pdflatex` (run twice), or with Tectonic, which performs reruns automatically. This copy was compiled from LaTeX using portable Tectonic 0.17.0 because `pdflatex` was unavailable. `report/results.tex` records the measured experiment values.

Part A. The submitted branch is `Yousef-Mohamed-Mahmoud-Helmy-Ismail_20P7954`. Open the completed assignment branch on GitHub. The trusted workflow grades only `src/engine.py` and `src/nn.py`. The course autograder runs when a pull request is opened against the course repository's main branch. LMS upload is a separate submission action.

Part B. The accompanying archive contains cleaned code, tests, reproduction instructions, the executed notebook, the compiled report and LaTeX source, plots, screenshots, and machine-readable verification evidence.

References and provenance

1. Ain Shams University, Faculty of Engineering. *CSE485-CSE488: Deep Learning, Assignment 1: MicroGrad (Scalar Autograd Engine)*. Supplied three-page assignment specification.

2. Completed submission: Yousef Mohamed Mahmoud Helmy Ismail – MicroGrad code and artifacts. GitHub account: `glacialhex`; student ID: `20P7954`.

3. Course template, README, source stubs, tests, and autograder workflow: DL-Course-ASU / assignment-1-microgrand-template. Inspected template commit `2a9d379546bdc61b8e3ff846e551051ac0602bc6`.

4. Andrej Karpathy. *micrograd*, the educational inspiration named in the assignment. karpathy / micrograd.

5. Python Software Foundation. Standard-library `math` functions. Python math documentation.

Implementation provenance. The submitted implementation fills the course template while preserving its public signatures and tanh behavior. The optional Module base class, reverse arithmetic operators, ReLU, dimension checks, and numerical regression tests extend the template within the assignment's scalar scope. No external automatic differentiation library is imported by the engine, network, verification script, or training notebook.

Tooling provenance. This submission was prepared with AI coding assistance. Code correctness is supported by the preserved course tests, additional independent checks, and executed experiment artifacts. The student can inspect and reproduce every saved result from the submitted sources.

Verification boundaries. The report's measurements come from the saved run, rather than theoretical guarantees. Raw CLI transcripts and JUnit results can be rerun; screenshots provide visual context. The rendered PDF was inspected page by page for layout and missing content. The package manifest provides SHA-256 hashes for integrity checks after extraction.